THREATTRUST

Complete Technical & Product Project Bible

Decentralized Cyber Threat Intelligence Sharing Platform

Original concept: Artificial Insaans — Decentralized Cyber Threat Intelligence Sharing Platform

Tagline: Detect Once. Defend Everywhere.

This is the master working specification for developing ThreatTrust from the original Artificial Insaans proposal. It covers the concept, problem, workflow, trust model, CTI model, data schema, blockchain architecture, application architecture, security, implementation, scope, roadmap, changes from the original idea, risks, and final specification.

1. Project Identity

Final name: ThreatTrust

Original concept: Artificial Insaans — Decentralized Cyber Threat Intelligence Sharing Platform

Category: Blockchain / Web3 / Cybersecurity

Core: Decentralized Cyber Threat Intelligence + decentralized trust between independent organizations

> **One-line definition:** ThreatTrust is a decentralized trust layer that enables verified organizations to validate, endorse, and securely share cyber-threat intelligence through a tamper-evident blockchain network.

The original proposal describes a permissioned decentralized trust network for inter-organizational cyber defense.

2. Detailed Problem

Organizations such as banks, government agencies, CERTs and enterprise SOCs continuously discover malicious indicators. Useful intelligence can include malicious IPs, domains, URLs, malware hashes, C2 domains and attack patterns.

The central problem is not simply collecting more threat data. Organizations need confidence that shared intelligence is authentic, useful, traceable and not manipulated. The original proposal identifies lack of trust, tampering, fake or unverified intelligence, contributor accountability, slow sharing and interoperability as key problems.

> **Core problem:** independent organizations need a common trust and audit mechanism for sharing CTI without making one organization the unquestioned owner of that shared trust record.

3. Who Uses ThreatTrust

Primary participants: banks and FinTech organizations; government organizations and CERTs; enterprise SOCs; and later, other critical-sector organizations.

MVP: represent these roles with 2–3 simulated organizations rather than attempting to build a production consortium.

4. Complete Concept and Workflow

ThreatTrust treats an Indicator of Compromise (IoC) as more than a piece of data. It becomes a shared threat record whose confidence can increase through independent validation and endorsement and whose history is auditable.

Step 1 — Organization identity: an authorized organization joins the network.

Step 2 — Detection: the organization detects a potentially malicious indicator.

Step 3 — Submission: the organization submits the IoC and metadata.

Step 4 — Normalization: the value is converted into a consistent representation before comparison.

Step 5 — Validation: the system checks type/format, duplicate status and contributor reputation.

Step 6 — Peer endorsement: other authorized organizations review and endorse the intelligence.

Step 7 — Smart contract/chaincode: defined rules determine whether the indicator can become verified and record the appropriate state.

Step 8 — Blockchain record: the relevant trust/integrity state is written to the permissioned ledger.

Step 9 — Distribution: verified intelligence becomes available to participating organizations through the application/API layer.

Step 10 — Reputation: contributor reputation changes based on the agreed outcome.

Step 11 — Audit/integrity: later changes can be checked against the blockchain-anchored integrity record.

5. Concrete Example

Participants: Bank A, Bank B and CERT C.

Bank A detects a malicious IP and submits it. ThreatTrust normalizes and validates the value, checks whether the same IoC already exists, and evaluates Bank A's reputation. Bank B and CERT C independently review and endorse the submission. Once the defined conditions are satisfied, the blockchain records the verified state and the intelligence becomes available to Bank B and CERT C.

If Bank A starts at reputation 72 and makes a validated contribution, the prototype can show **72** → **73**. A confirmed false submission can reduce the score according to the agreed penalty.

> An integrity check can detect an attempted modification and show **BLOCKCHAIN INTEGRITY CHECK FAILED**.

6. Why Blockchain?

A conventional database can store IoCs, so 'blockchain makes data immutable' is not sufficient justification. The stronger argument is that multiple independent organizations need a shared trust record without one organization being the unquestioned controller of it.

ThreatTrust uses a permissioned blockchain to provide a shared tamper-evident record, traceable contributor and endorsement history, auditable transactions, and enforceable smart-contract/chaincode rules.

> **Core positioning:** ThreatTrust is not merely putting cyber threats on a blockchain. It is using blockchain-backed trust to coordinate and audit CTI sharing between independent organizations.

7. Trust Model

Identity: participating organizations are authorized.

Reputation: contributor reliability changes according to contribution quality and validation history.

Peer endorsement: independent participants can confirm or challenge an indicator.

Ledger: important trust and integrity state is recorded in a tamper-evident blockchain.

Peer endorsement does not mean the network has discovered absolute truth; it represents the defined trust/verification state of the consortium.

8. CTI and STIX Model

The original proposal identifies STIX 2.1 threat indicators such as malware hashes, phishing URLs, malicious IPs, C2 domains and attack patterns.

MVP: start with IP, URL, domain and file hash. Keep the data model extensible for additional STIX objects and attack patterns.

Future integration: REST/TAXII-compatible exchange so the platform can eventually connect to existing CTI systems.

9. Data Schema

Organization: organization_id, name, type, status, identity/reference, reputation_score, created_at.

User: user_id, organization_id, role, authentication_reference, status.

IoC / Threat Record: ioc_id, ioc_type, normalized_value, contributor_org_id, created_at, status, confidence/trust_state, reputation_at_submission, integrity_hash/reference, evidence_reference.

Endorsement: endorsement_id, ioc_id, organization_id, decision, timestamp, reason/reference.

Reputation Event: event_id, organization_id, event_type, score_delta, related_ioc_id, timestamp.

Audit Event: event_id, actor/organization, action, object_id, timestamp, blockchain_transaction_reference.

10. On-Chain vs Off-Chain

On-chain candidates: IoC identity/reference, appropriate IoC representation or integrity hash, contributor organization reference, timestamps, status, endorsement/trust state, reputation-related state and integrity references.

Off-chain: large evidence files, detailed incident reports, sensitive context and other information that should not be directly exposed on-chain.

Privacy rule: before using any public-chain fallback, explicitly review whether raw IoC values themselves should be public. The permissioned Fabric model is a better fit for the original enterprise trust model.

11. Duplicate Detection

> **Exact MVP rule:** same normalized IoC type + same normalized IoC value = duplicate.

Example: IP + 185.10.20.30 submitted twice → one threat record.

If Bank A and Bank B independently report the same malicious IP, Bank B adds an observation/endorsement to the existing record rather than creating a second threat. This makes independent confirmation useful to the trust model.

Normalization rules should be defined per IoC type so trivial formatting differences do not bypass duplicate detection.

12. Reputation System

Initial MVP: validated contribution = +1; confirmed false/invalid contribution = –3; below a defined threshold = restricted submission rights.

Example: reputation 72 → valid contribution → 73. A later confirmed false contribution can reduce it according to the penalty.

The formula is intentionally simple for the prototype and can later be replaced by a richer model. It must be finalized early because it affects chaincode, backend, UI and testing.

13. Peer Endorsement

The submitting organization is the contributor. Other authorized organizations can review and endorse, reject or flag the indicator.

For the prototype, choose a simple verification threshold such as two independent endorsements. The exact threshold should be finalized before implementation.

14. Smart Contract / Chaincode Specification

registerOrganization() — create/authorize an organization record.

submitIoC() — create a pending IoC record.

checkDuplicate() — identify an existing normalized IoC.

validateIoC() — apply format and validation rules.

endorseIoC() — record an authorized peer decision.

verifyIoC() — transition an IoC to verified/rejected when conditions are satisfied.

updateReputation() — apply the defined score change.

flagIoC() — record a challenge or false-intelligence decision.

getThreat() — retrieve threat state.

verifyIntegrity() — compare current data/integrity information with the anchored record.

15. Blockchain Architecture

Preferred: Hyperledger Fabric permissioned network.

Conceptual flow: Organizations → API/Application → validation → chaincode → Fabric peers/orderer/ledger → blockchain event → backend → dashboards/API.

Fabric fits the original model because participants are verified organizations. The original proposal specifies Hyperledger Fabric and chaincode and describes Dockerized simulated nodes as a prototype approach.

Fabric rule: keep Fabric if the team can execute it reliably. If setup becomes the dominant time sink and threatens the working prototype, use Solidity + Hardhat/local EVM as an explicitly simplified prototype.

16. Application Architecture

Frontend: React or Next.js dashboard for organization views, threat feed, submission, endorsement, reputation and integrity screens.

Backend: Node.js or Python API for authentication, validation orchestration, database operations, blockchain calls and events.

Blockchain: Hyperledger Fabric + Go/Node chaincode.

Database: PostgreSQL or MongoDB for application/query data that does not require ledger semantics.

Storage: private/off-chain storage for evidence and large artifacts.

Standards: STIX 2.1 internally; REST/TAXII compatibility as the integration direction.

Security: RBAC and organization identity; production PKI can remain a roadmap item if not needed for the prototype.

17. Frontend Modules

Organization Dashboard: reputation, submitted indicators, endorsements, alerts and activity.

Submit IoC: type, value, optional context/evidence reference and submission action.

Threat Feed: verified and pending intelligence with status, confidence and source organization.

Threat Details: IoC, timeline, endorsements, trust state, contributor and integrity information.

Endorsement Panel: review and endorse/reject pending intelligence.

Reputation: current score and reputation events.

Audit/Integrity: transaction reference, hash/integrity state and verification result.

Organization/Admin: prototype organization registration and authorization controls.

18. Backend / API Responsibilities

Organizations: register, list, get profile, get reputation.

Threats: submit, list, get details, search/filter, validate.

Endorsements: endorse, reject/flag, list endorsements.

Reputation: get score, get history.

Integrity: verify record, retrieve blockchain reference.

Events: consume blockchain events and update application views.

19. Security Model

Identity: only authorized organizations perform consortium actions.

RBAC: contributor, reviewer/endorser and administrator roles have different permissions.

Integrity: blockchain-anchored hashes/references make unauthorized modification detectable.

Privacy: sensitive context and large evidence remain off-chain.

Accountability: submissions, endorsements and reputation events are attributable to organizations.

False intelligence: peer validation and reputation reduce the impact of low-quality contributors.

20. AI Decision

AI is not the core innovation. Do not add it simply for buzzwords.

If genuinely useful later, AI can support IoC normalization, correlation, prioritization or summarization. It remains a supporting layer.

> **Core innovation:** decentralized CTI + decentralized trust.

21. MVP Scope

Must work: 2–3 simulated organizations; organization identity; IoC submission; normalization; validation; duplicate detection; peer endorsement; reputation; smart contract/chaincode transaction; blockchain-backed record; shared dashboard; prototype propagation; integrity verification.

Do not prioritize: Splunk/Wazuh/ELK integrations, production PKI, large-scale cloud infrastructure, multi-sector production deployment and complex governance/DAO mechanisms.

22. Roadmap

CTI: more STIX objects, TAXII exchange, richer threat relationships and correlation.

Enterprise: SIEM/SOC integrations, production identity, organization onboarding and large-scale deployment.

Privacy: stronger selective disclosure and privacy-preserving intelligence sharing.

Trust: richer reputation, contributor history and confidence models.

Governance: consortium policies, contribution rules and more sophisticated decision mechanisms.

23. Vibe-Coding Implementation Plan

Phase 1 — Foundation: finalize schema, duplicate rules, reputation formula, roles and blockchain decision. Set up repository, Docker and development environments.

Phase 2 — UI: build dashboard, threat feed, submission, threat details and endorsement screens with mock data.

Phase 3 — Backend: implement API, validation, normalization, duplicate logic, reputation service and database.

Phase 4 — Blockchain: create Fabric network/chaincode or fallback EVM contract. Implement organization, IoC, endorsement, verification and reputation transactions.

Phase 5 — Integration: connect frontend → backend → blockchain and consume blockchain events.

Phase 6 — Integrity: implement hash/reference generation and real integrity verification.

Phase 7 — Testing: test duplicate submissions, endorsements, false submissions, reputation changes, permissions and integrity failures.

Phase 8 — Hardening: seed realistic data, improve UX, remove unnecessary features and document the architecture.

Vibe-coding rule: AI can generate scaffolding, UI, CRUD and boilerplate, but blockchain configuration, identity/permissions, transaction semantics and security-sensitive logic must be manually reviewed and tested.

24. Development Difficulty

Frontend: Medium.

Backend: Medium.

Database: Low–Medium.

Reputation/duplicate logic: Low–Medium, but must be precisely specified.

Fabric: High relative difficulty; network setup, identities, peers, ordering and chaincode integration are the main execution risks.

EVM smart contract: Medium.

Integration: Medium–High because multiple layers must agree on transaction state.

Overall: ambitious but feasible as a controlled prototype if scope is protected.

25. Changes From Original Artificial Insaans

The core idea remains the same. The changes make the original broad proposal more concrete, demoable and buildable.

Area	Original	Final
Core	Decentralized CTI sharing	Same; decentralized trust-layer framing
Participants	Banks, FinTech, government/CERTs, enterprise SOCs	Same model; 2–3 simulated organizations for MVP
IoCs	IP/domain, malware hash, TTP/attack pattern	Same; start with IP/URL/domain/hash
Workflow	Detect → Submit → Validate → Endorse → Share	Adds normalization, duplicate, reputation and integrity stages
Duplicate	Mentioned	Normalized type + value rule
Reputation	Dynamic score concept	Concrete +1 / −3 / threshold MVP model
Endorsement	Included	Core MVP trust step with defined threshold
Blockchain	Permissioned Fabric	Keep Fabric if executable; fallback only if execution risk is real
Integrity	Tamper-evident ledger	Actual integrity/tampering verification flow
On/off-chain	Sensitive/large data off-chain	Same; explicit data boundary
SIEM	Splunk/ELK/Wazuh	Roadmap, not MVP
PKI	Certificates/PKI	Production-grade implementation can be roadmap
AI	Not central	Do not force AI; optional support only
Data model	Broad proposal	Concrete organization/IoC/endorsement/reputation/audit entities
Implementation	Broad architecture	Phased implementation plan

26. Decisions to Lock Before Coding

1. Fabric vs EVM prototype.
2. Exact IoC fields.
3. Exact on-chain/off-chain boundary.
4. Normalization rules by IoC type.
5. Duplicate key.
6. Reputation formula and thresholds.
7. Endorsement threshold.
8. Organization/user roles.
9. Integrity/hash strategy.
10. MVP screens and API contract.

27. What We Should Not Build

Do not turn ThreatTrust into a generic cybersecurity dashboard. Do not add AI merely because it is fashionable. Do not build every SIEM integration. Do not implement complex enterprise PKI if it threatens the prototype. Do not put sensitive evidence directly on-chain. Do not create complicated governance before the core trust flow works.

Every feature should strengthen decentralized trust, improve CTI sharing, make blockchain useful, improve the prototype, or provide meaningful differentiation. Otherwise postpone it.

28. Main Risks and Mitigations

Novelty risk: CTI sharing and blockchain trust are not completely unexplored. **Mitigation:** differentiate through the concrete trust/endorsement/reputation model and implementation.

Fabric execution risk: Fabric can consume substantial setup time. **Mitigation:** time-box Fabric setup and maintain the simplified EVM fallback.

Scope risk: enterprise integrations can overwhelm the MVP. **Mitigation:** protect the end-to-end trust flow.

Trust-model ambiguity: vague rules create rework. **Mitigation:** lock schema, duplicate, reputation and endorsement rules before coding.

Demo reliability risk: multiple layers can fail. **Mitigation:** maintain pre-seeded transactions, logs/screenshots and a recorded fallback.

29. Honest Hackathon Assessment

These are qualitative ratings for the refined concept, not objective measurements or a guarantee of winning.

Criterion	Rating
Problem significance	9/10
Blockchain/Web3 relevance	9/10
Technical depth	9/10
Innovation	8/10
Differentiation	8/10
Demo potential	9/10
Long-term potential	9.5/10
Overall	9.0/10

Why it is strong: serious cybersecurity problem, meaningful decentralization, peer trust/reputation, technical depth and a clear product concept.

Main weakness: the underlying areas are not entirely new. Judges may challenge the necessity of blockchain and the differentiation. The trust-layer framing, concrete rules and actual implementation must answer that.

30. Final Locked Specification

Project: ThreatTrust

Original concept: Artificial Insaans — Decentralized Cyber Threat Intelligence Sharing Platform

Core: decentralized CTI + decentralized trust between independent organizations.

MVP: Submit → Normalize → Validate → Duplicate Check → Endorse → Record → Share → Reputation → Integrity Check.

Preferred blockchain: Hyperledger Fabric, provided the team can execute it reliably.

Fallback: Solidity + Hardhat/local EVM only if Fabric execution risk threatens the working prototype.

AI: optional supporting capability only; never the headline innovation.

> **Final definition:** ThreatTrust is a decentralized trust layer that enables verified organizations to validate, endorse, and securely share cyber-threat intelligence through a tamper-evident blockchain network.

Status: This is the master project direction to follow. The next stage is implementation planning and architecture rather than further idea selection.